



**AI Driven SQA: Manual & Automation Testing**

## **Assignment 04(02)**

**Module: 04**

**Topic: Introduction to Programming Language  
with AI (JavaScript + Node.js).**

**Submitted by,**

**Name: Jhion Adbar Opi**

**Batch: 21**

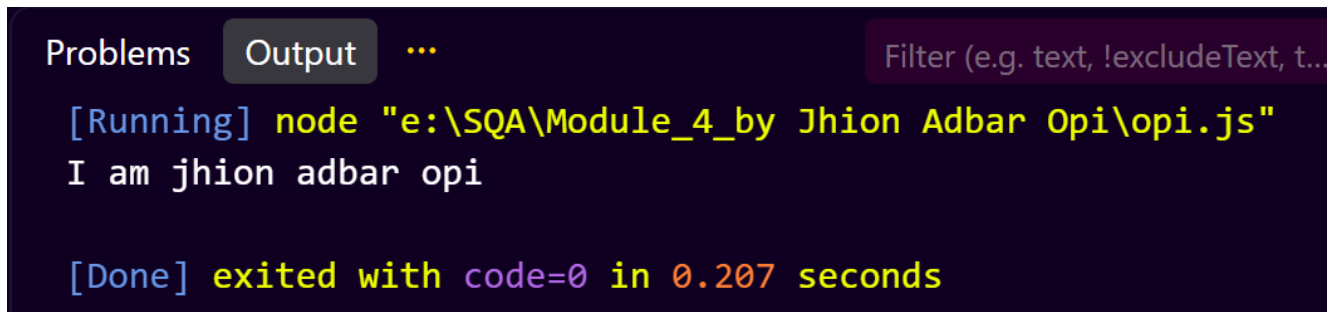
**Phone: 01778812373**

## Q1. Introduction to Node.js

### 01. What is Node.js?

**Answer:** Node.js is a server-side runtime environment that executes JavaScript code outside the browser and provides various built-in modules for developing web applications, APIs, real-time systems, and network applications.

```
Example: console.log("I am Jhion Adbar Opi");
```



```
Problems Output ... Filter (e.g. text, !excludeText, t...)
[Running] node "e:\SQA\Module_4_by Jhion Adbar Opi\opi.js"
I am Jhion Adbar Opi

[Done] exited with code=0 in 0.207 seconds
```

### 02. How does Node.js work?

**Answer:** Node.js works using an event-driven, non-blocking I/O architecture. Unlike traditional server technologies that create a new thread for each client request, Node.js uses a single-threaded event loop to handle multiple requests efficiently. This architecture allows Node.js to process thousands of concurrent connections while consuming fewer system resources.

#### Step 1: Client Sends a Request

A user sends a request to the Node.js server.

Examples:

- Opening a website
- Requesting data from an API
- Uploading a file

**// Client => Node.js Server**

#### Step 2: Request Enters the Event Queue

Node.js receives the request and places it in the Event Queue. The Event Queue stores all incoming tasks waiting to be processed.

**// Client Request => Event Queue**

#### Step 3: Event Loop Checks the Queue

The Event Loop continuously monitors the Event Queue. It checks whether new tasks are available and decides how they should be processed.

**// Event Queue => Event Loop**

The Event Loop is the core component of Node.js.

Jhion Adbar Opi

#### Step 4: Non-Blocking Execution

If a task requires a database query, file operation, or network request, Node.js does not wait for the operation to finish.

##### Instead:

- The task is assigned to the system.
- Node.js continues processing other requests.
- When the operation completes, a callback function is executed.

This behavior is called non-blocking I/O.

#### Step 5: Callback Execution

After the background operation finishes, Node.js places the callback function back into the Event Queue. The Event Loop then executes it.

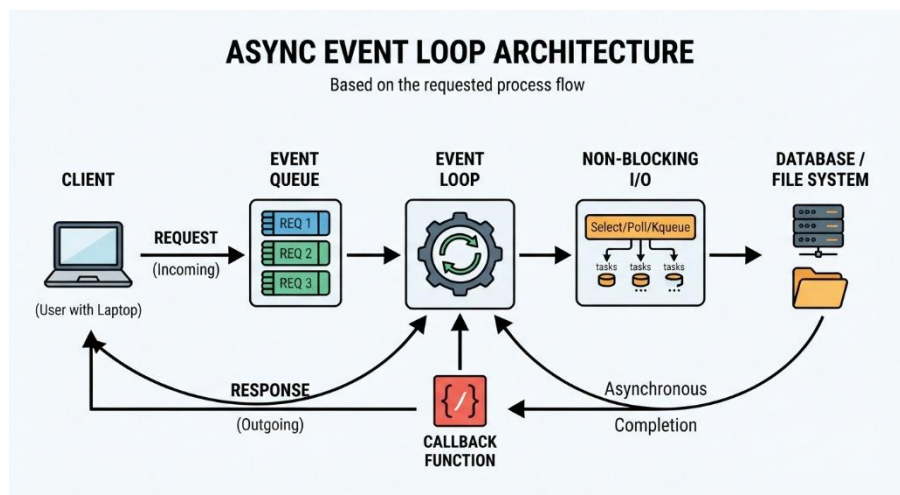
**// File Read Complete => Callback Queue => Event Loop**

#### Step 6: Response Sent to Client

Finally, Node.js sends the processed result back to the user.

**// Node.js Server => Response => Client**

### Node.js Working Architecture



## Example:

```
const fs = require("fs");

console.log("Start");

fs.readFile("data.txt", "utf8", (err, data) => {
  console.log(data);
});

console.log("End");
```

```
[Running] node "e:\SQA\Module_4_by Jhion Adbar Opi\tempCodeRunnerFile.js"
Start
End
undefined

[Done] exited with code=0 in 4.197 seconds
```

### Q3. Advantages of Node.js for QA Automation.

#### Answer:

Node.js technology in Quality Assurance (QA) automation because it provides a fast, efficient and scalable environment for developing and executing automated tests. Many modern testing frameworks as such as Selenium WebDriver, Playwright, Cypress and WebdriverIO are built using JavaScript and Node.js, making it an essential platform for test automation.

Node.js provides numerous advantages for QA automation, including high performance, asynchronous execution, extensive testing framework support, CI/CD integration and cross-platform compatibility. Its ability to execute automated tests efficiently and support modern testing tools makes Node.js one of the most widely used technologies for developing scalable and reliable QA automation solutions.

| Advantage                                   | Description                                                                                                                                                                                                                                                                                      |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fast Test Execution                         | Node.js is built on Google's V8 JavaScript Engine, which compiles JavaScript into machine code and executes it efficiently. This results in faster test execution and reduced testing time, especially for large test suites.                                                                    |
| Asynchronous and Non-Blocking Architecture  | Node.js uses an asynchronous, non-blocking I/O model that allows multiple testing operations to run simultaneously without waiting for previous tasks to complete. This improves automation efficiency and enables faster validation of applications.                                            |
| Single Language for Development and Testing | Since many web applications use JavaScript on the frontend, QA engineers can use the same language for both application development and test automation. This reduces the learning curve and improves collaboration between developers and testers.                                              |
| Rich Testing Framework Ecosystem            | Node.js provides access to the Node Package Manager (NPM), which contains thousands of testing libraries and tools. Popular QA automation frameworks such as Playwright, Cypress, Selenium WebDriver, Mocha, Jest, and WebdriverIO can be easily installed and integrated into testing projects. |
| Easy Integration with CI/CD Pipelines       | Node.js integrates seamlessly with Continuous Integration and Continuous Deployment (CI/CD) tools such as Jenkins, GitHub Actions, GitLab CI/CD, and Azure DevOps. Automated tests can be executed whenever code changes are committed, supporting faster and more reliable software delivery.   |
| Cross-Platform Compatibility                | Node.js applications and automation scripts can run on Windows, Linux, and macOS with minimal modifications. This enables QA teams to test applications across different environments efficiently.                                                                                               |
| Support for Modern Web Application Testing  | Modern web applications often use dynamic technologies such as React, Angular, and Vue.js. Node.js-based automation tools are specifically designed to handle these modern web technologies, providing accurate and reliable test results.                                                       |
| Scalability and Parallel Testing            | Node.js supports parallel execution of automated tests, allowing multiple test cases to run simultaneously. This significantly reduces overall testing time and improves productivity in large-scale software projects.                                                                          |
| Large Community and Active Support          | Node.js has a large global community that continuously develops new tools, plugins, and best practices. QA engineers can easily find documentation, tutorials, and solutions to automation challenges.                                                                                           |
| Cost-Effective Automation Solution          | As an open-source platform, Node.js can be used without licensing costs. Organizations can build powerful automation frameworks using freely available tools and libraries, reducing overall testing expenses.                                                                                   |

So, Node.js is the best .

#### Q4. Difference between Browser JavaScript and Node.js.

Answer:

| Feature                           | Browser JavaScript                                                                                     | Node.js                                                                                      |
|-----------------------------------|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>Execution Environment</b>      | Runs inside a web browser such as Chrome, Firefox, or Edge.                                            | Runs outside the browser on a server or computer.                                            |
| <b>Purpose</b>                    | Used to create interactive and dynamic web pages.                                                      | Used to build server-side applications, APIs, and backend services.                          |
| <b>Access to System Resources</b> | Limited access for security reasons. Cannot directly access local files or operating system resources. | Can access files, operating system resources, and network services through built-in modules. |
| <b>Global Object</b>              | Uses window as the global object.                                                                      | Uses global as the global object.                                                            |
| <b>Modules</b>                    | Traditionally does not include built-in server modules.                                                | Provides built-in modules such as fs, http, path, and os.                                    |
| <b>File System Access</b>         | Cannot directly read or write files on the local machine.                                              | Can read, write, and manage files using the File System (fs) module.                         |
| <b>Package Management</b>         | Relies mainly on browser libraries and frameworks.                                                     | Uses NPM (Node Package Manager) to install and manage packages.                              |
| <b>API Support</b>                | Uses Browser APIs such as DOM, BOM, and Local Storage.                                                 | Uses Node.js APIs for server-side operations and system interaction.                         |
| <b>Typical Applications</b>       | Interactive websites, form validation, animations, and user interface behavior.                        | Web servers, REST APIs, real-time applications, microservices, and automation tools.         |
| <b>Examples</b>                   | Button click handling, form validation, image sliders.                                                 | Web servers, chat applications, backend services, and QA automation frameworks.              |

Browser JavaScript and Node.js both use the JavaScript programming language. But they serve different purposes. Browser JavaScript is mainly used for client-side web development and user interaction. While Node.js is used for server-side development, backend services and system-level operations. Node.js extends JavaScript beyond the browser by providing access to operating system resources, file handling, networking capabilities, and package management through NPM.